

CURVE

MVP & Full Product Specification

A detailed blueprint for a local-first AI execution workspace that evolves into a multi-agent, multi-device execution platform.

Version: 1.0 — October 2026

Document type: Product definition / founder build specification

CORE PRINCIPLE

Curve does not need to own all compute. Curve needs to orchestrate compute: understand an outcome, plan the work, obtain permission, execute through the right environment, verify the result, and turn successful work into reusable workflows.

This document separates the launchable MVP from the larger Curve vision so the product can be built with very little money instead of prematurely becoming a cloud-compute company.

1. Executive definition

Curve is an AI execution workspace. A user gives Curve an outcome instead of manually performing every step. Curve turns the outcome into a plan, assigns work to agents/tools, asks for approval when required, executes inside a bounded environment, verifies what happened, and returns artifacts plus a concise activity record.

The first version should not attempt to beat frontier labs at raw model intelligence or reproduce expensive cloud-computer infrastructure. It should win by building a better **orchestration and execution layer** around models the founder can already access.

One-sentence product

Curve turns goals into controlled, verifiable work by coordinating AI agents, tools, and execution environments.

Three layers

Layer	Purpose	MVP
Intelligence	Interpret goals, plan steps, select tools, reason and review	Yes — one model provider initially
Orchestration	Projects, agents, workflows, permissions, state, approvals, verification	Yes — core differentiator
Execution	Actually touch files, terminal, browser and other allowed resources	Yes — local Curve Runner

What Curve is not

- Not merely a chatbot with a fancy UI.
- Not merely a coding agent.
- Not an unrestricted remote-control system for a user’s entire computer.
- Not a frontier foundation-model company in the MVP.
- Not a cloud-compute business on day one.

2. Product thesis and positioning

The problem

AI can be capable while still forcing users to manage context, tools, permissions, environments and follow-up manually. Curve makes **execution itself** the product: the user describes the result; Curve manages the work required to get there.

Core loop

Intent → Understand → Plan → Permission check → Execute → Observe → Verify → Report → Remember → Reuse

Positioning

Category	Emphasis	Curve answer
Chat assistant	Conversation and answers	Useful, but insufficient for multi-step work
Coding agent	Repository changes	One important Curve agent, not the whole product
Computer agent	Operate a computer	One execution capability, not the whole product
Automation platform	Rules/integrations	Curve adds adaptive reasoning + agents
AI workspace	Projects/collaboration	Curve adds actual bounded execution

Differentiation principle

Curve should not claim that its model is smarter than every competitor. The differentiation is the **system** around the model: project context, reusable workflows, role-based agents, explicit permissions, execution routing and verification.

3. What Curve does

MVP capabilities

Capability	Concrete behavior
Projects	Persistent workspace containing goals, files, agents, workflows, permissions, runs and artifacts.
Goal input	User describes an outcome in natural language.
Task planning	Curve turns the goal into ordered, inspectable steps before execution.
Approval gates	Curve pauses before sensitive, destructive or externally visible actions.
Local execution	Curve Runner executes approved actions on the user's machine.
File operations	Read, create, edit, rename and organize files inside allowed project scope.
Terminal	Run approved commands with timeouts, output capture and working-directory restrictions.
Search	Search project files before editing.
Git awareness	Inspect status/diff and optionally create commits after approval.
Verification	Run tests/checks and inspect outputs before declaring completion.
Artifacts	Collect changed files, reports, logs, screenshots and outputs.
Run history	Timeline of plans, actions, approvals and results.
Reusable workflows	Turn successful work into repeatable workflow templates.
Agent roles	Coder, Researcher, Reviewer, Planner and later more.
Model abstraction	Keep model calls behind a provider interface.

Future capabilities

- Browser execution and web research.
- Mobile Runner for supported phone/tablet actions.
- Cloud Runner for work that must continue when the PC is offline.
- Execution Router that chooses local, browser, mobile or cloud environments.
- Agent marketplace/library.
- Visual workflow builder.
- Multi-agent projects and teams.
- Scheduled long-running jobs.
- Team workspaces and shared workflows.
- Fine-tuned Curve-specific models.
- Cross-device handoff: start on desktop, monitor from phone.
- Usage-based cloud execution credits.
- Developer SDK/API for Curve-compatible agents and runners.

4. The actual MVP

The MVP exists to prove one proposition: **people will trust Curve to turn a natural-language goal into useful, verifiable work inside a project.**

MVP v0.1 — mandatory

Module	Definition of done
Web app	Create project, chat with Curve, inspect plan, approve actions and view results.
Curve Orchestrator	Maintains task loop, tool calls, state and model interaction.
Curve Runner	Local program accepts authenticated commands and performs bounded filesystem/terminal actions.
Project workspace	Files, settings, permissions, run history and artifacts are project-scoped.
Planner	Produces structured steps with tool requirements and risk levels.
Tool layer	read, write, edit, search, command, status/diff.
Approval system	Approve, reject or edit planned actions.
Verifier	Checks expected outputs and can request another model turn.
Run timeline	Shows what Curve planned, executed and observed.
Coding agent	Inspect repository, edit, run checks and present a diff.
Workflow save	Save a successful task as a reusable template.

MVP v0.2

- GitHub repository import/export.
- Controlled browser research.
- Project-level environment configuration.
- Agent roles and role switching.
- Retry/recovery logic.
- Run cancellation and resume.
- Workflow parameters.
- Local SQLite state and encrypted local secrets.

MVP v0.3 beta

- Polished onboarding.
- Permission editor.
- Workflow library.
- Better error explanations.
- Consent-based telemetry.
- Privacy-preserving crash reporting.
- One-click Curve Runner installer.

5. User experience

Main navigation

Screen	Purpose
Home	Projects, recent runs, saved workflows and quick-start goals.
Project	Central workspace for one ongoing objective.
Run	Live execution timeline, plan, approvals, tool results and artifacts.
Agents	Create/edit roles, instructions, tools and permissions.
Workflows	Saved reusable task recipes with parameters.
Artifacts	Files/results produced by Curve.
Settings	Model provider, Runner, permissions, privacy and account settings.

Project screen

- Left rail: Overview, Files, Runs, Agents, Workflows, Settings.
- Center: conversation/task composer and active work.
- Right panel: current plan, execution state, approvals and artifacts.
- Persistent status: Runner connected/disconnected, project permissions and model status.

Example interaction

User: “Fix the login bug, run the tests, and show me what changed.”

Curve plan: 1) inspect repository; 2) find auth code; 3) reproduce/identify failure; 4) edit; 5) test; 6) review diff; 7) report.

The user can expand a step to inspect the proposed action. Approval is requested for actions such as deleting files, pushing code, installing packages, accessing paths outside the project or publishing externally.

Design rule: never make the user wonder what Curve is doing. The UI should expose intent, current step, permissions and result without dumping hidden model reasoning.

6. Curve Runner

Curve Runner is the key MVP infrastructure component: a small local application that provides controlled execution capabilities to Curve. The model does not directly own the user’s computer; the Runner is the enforcement boundary.

Runner responsibilities

- Authenticate with Curve.
- Receive structured tool requests.
- Validate every request against local/project permissions.
- Execute allowed operations.
- Capture stdout/stderr, exit codes and metadata.
- Apply timeouts and cancellation.
- Return structured results.
- Reconnect safely.
- Keep secrets out of model-visible workspace data where possible.

Initial tool contract

Tool	Input	Output
read_file	workspace-relative path	text + metadata
write_file	path + content	success + metadata
apply_edit	path + patch	success + diff
search_files	query + scope	matching paths/snippets
run_command	command + cwd + timeout	stdout/stderr/exit code
list_files	directory	entries + metadata
git_status	repo path	status
git_diff	repo path	diff
create_commit	message + repo	commit hash; approval required

Boundary

Default: Curve can operate only inside the selected project directory. Access outside it is denied unless explicitly allowlisted. Network access is disabled or tightly restricted for commands that do not need it.

Architecture

Web UI → Curve backend/orchestrator → authenticated Runner channel → local policy engine → tool executor → result stream

Modern agent systems increasingly separate orchestration/control from execution. OpenAI’s current sandbox documentation describes a control-plane harness versus execution-plane sandbox, while its self-hosted environment docs describe an executor running on a laptop, container or remote environment.

`[cite[turn0search0][turn0search1]`

7. Agent system

Agents are reusable roles with explicit capabilities and responsibilities. Curve should not force every task through one giant prompt.

Agent	Job	Typical tools
Planner	Understand goal, decompose work, coordinate execution	Planning metadata; limited execution
Coder	Modify software and run checks	Filesystem, search, terminal, git
Researcher	Gather and synthesize information	Browser/web later; files
Reviewer	Inspect output for correctness	Read/search/diff/test results
Writer	Produce structured documents	Files + research
Data Analyst	Analyze data and generate reports	Files + Python/terminal

Agent definition

Each agent has: name, purpose, system instructions, allowed tools, default permission level, input/output schema, preferred model, memory scope and verification rules.

Lifecycle

Create → Configure → Test → Save → Run → Review → Improve → Version

Multi-agent future

A project might form a team: Researcher gathers information → Analyst processes it → Writer produces the report → Reviewer checks it → Coordinator packages the artifact. Each role gets only the permissions it needs.

Important: many agents are not the moat. Coordination, permissions, state and verification are.

8. Workflow system

A workflow is a reusable description of how Curve performs a class of task. Unlike a simple macro, it can contain conditional steps, agent roles, tools, approvals and verification.

Workflow structure

Trigger → Inputs → Plan → Agent steps → Conditions → Approvals → Execution → Verification → Outputs → Memory update

Example: fix and ship a website change

Step	Action	Approval
1	Load repository + instructions	No
2	Inspect issue/files	No
3	Implement change	No, inside project
4	Run tests/build	No
5	Review diff	No
6	Create commit	Yes
7	Push remote	Yes
8	Deploy	Yes
9	Verify deployment	Depends

Parameters

- repository
- branch
- test command
- build command
- output location
- review strictness
- approval mode
- agent/model preference

Users should be able to save successful work as recipes and rerun them later. This turns Curve from a one-off agent into a system for repeatable work.

9. Permissions, approvals and trust

Permissioning is a visible product feature, not just a hidden security implementation.

Level	Examples	Default
Read-only	Read/search/inspect	Allowed
Project write	Create/edit project files	Allowed with policy
Local execution	Run commands in project	Allowed with restrictions
Network	Reach approved domains	Ask
External write	Push/publish/send	Ask
Destructive	Large delete/reset	Ask every time
Sensitive access	Personal folders/credentials	Denied

Approval card

Show **what** Curve wants to do, **where**, **why**, expected impact and scope. Buttons: Allow once / Allow for workflow / Deny / Edit permission.

Security principles

- Least privilege by default.
- Project-relative paths.
- No secrets in prompts/logs/artifacts.
- Credentials remain runtime configuration where possible.
- Tool inputs are schema-validated.
- Commands have timeouts/output limits.
- Network access is allowlisted where feasible.
- Every action has an audit record.
- Runner makes final permission decision.
- Irreversible actions require explicit approval.

Current sandbox guidance similarly emphasizes scoped mounts, runtime credential handling and keeping sensitive control-plane data separate from model-directed execution. [cite]turn0search0[

10. Memory and project state

Memory	Contains	Scope
Conversation	Recent discussion	Conversation
Project	Stable facts/conventions/decisions	Project
Workflow	Lessons about a procedure	Workflow
Agent	Role-specific preferences/instructions	Agent

Example project memory

“This project uses Vite + React; npm run build is the production check; never modify /public/generated; deployment is manual and requires approval.”

Controls

- View memory.
- Edit memory.
- Delete memory.
- Disable memory for a project.
- Mark information temporary.
- Exclude folders from memory sources.

Do not dump every conversation into context. Memory should be summarized, searchable and selectively injected. Long-term, Curve can learn which workflows work reliably and which project conventions matter, while retaining only information the product is designed to retain.

11. Curve Coding Agent

The coding agent is the first flagship use case because it demonstrates the complete Curve loop: inspect → plan → edit → execute → test → verify → diff → report.

Capabilities

- Understand repository structure.
- Read relevant files before editing.
- Search symbols/text.
- Create/edit/delete within project permissions.
- Run tests, linters and builds.
- Inspect errors and iterate.
- Review git diff.
- Explain exactly what changed.
- Generate tests when useful.
- Stop when requirements are ambiguous.
- Save successful fixes as workflows.

State machine

INTAKE → REPO DISCOVERY → PLAN → EDIT → CHECK → TEST → FIX LOOP → REVIEW → APPROVAL → FINALIZE

Failure	Response
Command fails	Capture output; diagnose; retry if safe.
Tests fail	Inspect; correct; rerun.
Repeated failure	Stop after retry budget and ask user.
Ambiguous requirement	Ask targeted question.
Permission denied	Explain missing permission.
Unexpected change	Stop and surface diff.

The coding agent is one role inside Curve. Later the same project can use Researcher → Coder → Reviewer → Writer under one permission, workflow and run-history system.

12. Technical architecture

Recommended MVP shape:

Browser UI
↓
Curve API / Orchestrator
■■ Task Planner
■■ Agent Runtime
■■ Tool Router
■■ Permission Engine
■■ Verification Engine
■■ Project/Run State
■■ Model Provider Adapter
↓
Curve Runner (local)
■■ Filesystem tools
■■ Terminal tool
■■ Git tools

Component	MVP choice	Reason
Frontend	React + Vite or Next.js	Fast iteration
Backend	Python FastAPI or Node/TypeScript	Simple orchestration API
Runner	Python first; Rust/Go later	Fastest prototype
Database	SQLite	Zero-cost persistence
Model	One provider initially	Avoid complexity
Hosting	Low-cost web/serverless	Keep burn low
Packaging	Desktop installer later	Easy Runner onboarding

Provider abstraction

Define model.generate(messages, tools, schema, context) → response so the rest of Curve is provider-independent.

Execution abstraction

Define executor.read_file(...), executor.run_command(...), executor.apply_edit(...), executor.get_status(...). Implement LocalExecutor first; later add BrowserExecutor, MobileExecutor and CloudExecutor.

13. Data model

Entity	Important fields
User	id, preferences, model settings, privacy
Project	id, name, root, description, permissions, memory
Agent	id, role, instructions, tools, permissions, model, version
Workflow	id, name, inputs, steps, agents, conditions, approvals
Run	id, project, task/workflow, status, timestamps, model usage, executor
Step	id, run, description, tool, inputs, approval, status, result
Artifact	id, run, path/type, checksum, metadata
Permission	scope, resource, action, mode, expiration
MemoryItem	scope, content, source, confidence, timestamp
Runner	id, device, version, status, capabilities

Run states

QUEUED → PLANNING → WAITING_APPROVAL → EXECUTING → VERIFYING → COMPLETED / FAILED / CANCELLED / BLOCKED

Audit record

Every tool call records timestamp, actor/agent, tool, sanitized inputs, permission decision, execution environment, outcome, duration and artifact references.

14. End-to-end execution lifecycle

1. Intake

Classify the request: answer-only, workflow, coding, research, file task or multi-agent task.

2. Context collection

Load only relevant project context, files, memory and instructions.

3. Planning

Generate structured steps, expected outputs, required tools and risk levels.

4. Policy evaluation

Permission engine evaluates every capability. Safe steps proceed; sensitive steps become approval gates.

5. Execution

Runner performs bounded tool calls and returns results.

6. Observation

Inspect exit codes, diffs, generated files and other evidence.

7. Verification

Compare actual state to expected outcome; request more work if needed.

8. Completion

Summarize what happened and link artifacts, separating verified facts from assumptions.

9. Learning

Optionally save workflow and update project memory.

10. Reuse

Next time, run the workflow with new inputs rather than rediscovering the procedure.

This control-plane/execution-plane model matches current sandbox-agent architecture patterns.

`[cite[turn0search0][turn0search1]`

15. Example use cases

A — Coding

User: “Fix the broken dashboard and make sure the tests pass.”
Curve: inspect → identify → edit → test → inspect diff → verify → report.

B — Research

Future: Researcher gathers sources → Analyst structures comparison → Writer creates document → Reviewer checks it → artifact saved.

C — Repetitive workflow

User performs a cleaning/reporting task once → Curve saves it → future CSVs become inputs → same workflow runs.

D — Personal project assistant

“Continue where we left off” → Curve loads project state, memory and unfinished tasks → proposes next steps.

E — Multi-agent project

“Turn this idea into a technical proposal” → Planner → Researcher → Architect → Writer → Reviewer.

F — Cross-device future

Desktop handles code; phone monitors status; cloud continues when PC is offline if the user has enabled paid cloud execution.

16. MVP build plan

Phase	Build	Exit criterion
0 Foundation	Repo, UI, backend, Runner handshake	Browser communicates with Runner
1 Tools	read/write/search/list/command	Safe project manipulation
2 Planner	Structured plans + tool selection	Goal becomes steps
3 Approval	Permission engine + approval cards	Sensitive actions pause
4 Coding agent	Repo edits + tests + diff	Small coding tasks complete
5 Verification	Tests + diff review	No blind success claims
6 Persistence	SQLite projects/runs/artifacts	Work survives restart
7 Workflows	Save/replay parameterized workflows	Successful work reusable
8 Beta	Installer, onboarding, privacy	Small group can self-serve

Build order

Runner first → one tool → one model loop → multiple tools → planner → approvals → verification → coding agent → persistence → workflows → polish.

Do not build before the core loop

- Mobile app.
- Cloud computer fleet.
- Agent marketplace.
- Custom foundation model.
- Complex collaboration.
- Dozens of integrations.
- Fancy workflow canvas.
- Multi-provider model support.

17. Testing and success metrics

Functional acceptance tests

- Runner rejects unauthorized paths.
- Runner rejects commands outside policy.
- Curve edits a file and reports diff.
- Curve captures test failures.
- Curve uses failure output to make bounded corrections.
- Curve stops after retry budget.
- Curve asks before configured sensitive actions.
- A run can be cancelled.
- Run state can be recovered.
- Workflow can be saved and rerun with new parameters.

Product metrics

Metric	Why
Task completion rate	Does Curve accomplish requested work?
Verified completion rate	How often does verification agree?
Human intervention rate	How much supervision is needed?
Approval acceptance rate	Are prompts understandable?
Time to useful result	Is Curve faster than manual work?
Workflow reuse rate	Do saved procedures matter?
7/30-day retention	Does Curve become habitual?
Cost per completed task	Critical for future cloud economics.

North-star signal: verified work completed per active user per week.

18. Business model

MVP economics

Use the user’s computer for execution wherever possible. The main variable cost is model usage plus lightweight orchestration hosting.

Tier	Concept
Free	Limited projects, local execution, basic agents/workflows.
Pro	Higher usage, advanced agents, browser tools, priority execution.
Team	Shared projects, permissions, audit logs, shared workflows.
Enterprise	Governance, SSO, dedicated environments and compliance features.
Cloud Credits	Optional paid credits for cloud-run jobs.
Developer/API	Usage-based API for Curve-compatible execution.

Potential moat

- Reusable workflow library.
- Project state and execution history.
- Permission model.
- Execution abstraction across environments.
- Agent coordination.
- Verification/evaluation data.
- Developer ecosystem.
- Eventually, opt-in execution traces for Curve-specific model tuning.

19. Funding and infrastructure strategy

Stage 1 — ₹0 validation

- Local Runner.
- Low-cost web frontend.
- Minimal backend.
- One model provider.
- Your own machine for testing.
- Tiny group of testers.

Stage 2 — prove usage

- Measure completed tasks.
- Collect structured feedback.
- Improve reliability/onboarding.
- Create a five-minute demo.
- Avoid cloud execution unless necessary.

Stage 3 — external support

Once demand is evidenced, investigate accelerators, grants, startup programs, angels and cloud credits. Current program terms should be checked when applying.

Stage 4 — cloud execution

Introduce CloudExecutor when there is a concrete reason: PC offline, hours-long jobs, standardized environment, or unavailable local compute.

Funding principle: do not raise money to discover whether Curve is useful; raise to scale something users have already demonstrated they want.

20. Phase 2 — Connected Curve

Browser execution

- Controlled browser runner.
- Research tasks.
- Web forms.
- Source collection.
- Page interaction with approvals.

Developer workflows

- Repository import.
- Issue-to-task workflows.
- Pull request preparation.
- Automated test/review pipelines.
- Optional push/deploy approvals.

Visual workflow builder

A canvas connects triggers, agents, tools, conditions, approvals and outputs. The workflow remains structured data so it can be versioned and shared.

Agent marketplace

Users install/create specialized agents. Every agent declares tools, permissions, inputs, outputs and version. Marketplace agents never bypass the project permission system.

21. Phase 3 — Multi-device Curve

Mobile Runner

A phone/tablet app can act as a supported execution endpoint. It should respect platform permissions and must not pretend to have unrestricted access to mobile functions.

Execution Router

Condition	Preferred executor
PC online + local files	Local Runner
Website interaction	Browser Runner
Supported mobile task	Mobile Runner
PC offline + cloud enabled	Cloud Runner
Heavy compute	Cloud/specialized compute

Cloud fallback

Policy examples: “Use local when available; otherwise ask before cloud.” Cloud becomes an explicit cost/privacy decision rather than a hidden dependency.

22. Long-term Curve vision

The eventual Curve is an **AI operating layer for work**: projects contain goals; goals activate workflows; workflows coordinate agents; agents use tools; tools execute through devices or cloud; results are verified and remembered.

CURVE
■ Intelligence Layer
■ General models
■ Curve models
■ Planner
■ Verifier
■ Tool/agent selection
■ Orchestration Layer
■ Projects
■ Goals
■ Agents
■ Workflows
■ Memory
■ Permissions
■ Audit/observability
■ Execution Layer
■ Local Runner
■ Browser Runner
■ Mobile Runner
■ Cloud Runner
■ Specialized compute

Super-agent concept

A future super-agent should be an orchestrated system that dynamically assembles specialized agents and tools while maintaining one objective, budget, permission policy and verification loop.

AI workforce framing

Users think in outcomes and roles: “research this,” “build this,” “analyze this,” “review this,” “keep this project moving.” Curve handles decomposition and execution boundaries.

23. Explicit non-goals

Do not build now	Reason
Frontier model from scratch	Unnecessary and prohibitively expensive for validation.
Unrestricted computer control	Poor security boundary.
Full cloud-computer fleet	High cost before validation.
100 integrations	Complexity before core value.
Perfect autonomy	Impossible target; focus on bounded verified work.
Large enterprise suite	No reason before product-market evidence.
Social network for agents	Distraction.
Complex marketplace	Wait until reusable agents are demanded.
Always-on autonomy	Add only after trust, scheduling and cost controls.

MVP rule: if a feature does not help prove Curve can reliably turn an outcome into verified work, postpone it.

24. Competitive positioning

Modern agent infrastructure is rapidly converging on model + tools + execution environments. OpenAI’s current sandbox documentation describes files, commands, packages, ports, snapshots and resumable state; its self-hosted environment docs describe an executor on a laptop/container/remote environment. Cloudflare’s current coding-agent tutorial likewise demonstrates an agent operating in a Linux sandbox with restricted outbound access. [\[cite\]](#)[turn0search0](#)[turn0search1](#)[turn0search3](#)

Weak positioning

“Curve lets an AI edit files and run terminal commands.” That is becoming standard infrastructure.

Stronger positioning

Curve is the orchestration layer where users manage AI workers, projects, workflows, permissions and execution environments.

Dimension	Curve ambition
Agent	Many specialized agents
Workspace	Project-centric and persistent
Execution	Local first; environment-agnostic later
Workflow	Reusable + parameterized
Permissions	First-class and user-visible
Verification	Built into completion
Memory	Project/workflow/agent state
Model	Provider-independent
Devices	Desktop → browser/mobile/cloud

Do not reproduce every competitor’s infrastructure. Build the layer that can use different execution providers underneath it.

25. Risks and mitigations

Risk	Impact	Mitigation
Bad edits	High	Tests, diff review, verification, retry limits.
Runner security hole	Critical	Least privilege, policy engine, isolation, approvals.
Cloud costs	High	Local-first, budgets, paid cloud credits.
Low user trust	High	Transparent plans, timelines, approval gates.
Product too broad	High	Coding/project workflow wedge first.
Competitors copy features	High	Focus on orchestration, workflows, execution abstraction.
Provider dependency	Medium	Provider adapter architecture.
Infinite agent loops	Medium	Max turns, timeouts, retry budgets.
Data exposure	Critical	Scoped access, secret isolation, privacy controls.
Founder builds too much	High	Ship execution loop before platform features.

Reliability principle: Curve should prefer “I could not verify this, so I stopped” over confidently reporting an unverified success.

26. Founder build checklist

Before coding

- Write MVP acceptance tests.
- Choose one model provider.
- Choose frontend/backend stack.
- Define Runner protocol.
- Define tool schemas.
- Define permission levels.
- Define Run/Step states.

Milestone 1

User opens Curve → creates project → connects Runner → says “create hello.txt containing Hello Curve” → Curve plans → Runner executes → UI shows action → artifact appears.

Milestone 2

“Build a tiny website in this folder” → Curve creates files → runs checks → reports result.

Milestone 3

Existing repository + bug → search → edit → test → iterate → show diff → ask before external action.

Milestone 4

Save that process as a workflow → rerun with a new input → Curve performs the same class of work.

Do not move to cloud until

- People complete real tasks.
- Some users return voluntarily.
- You know which tasks are valuable.
- Task success is measurable.
- Average model cost per successful task is understood.

27. Appendix — technical interfaces

Runner request

```
{
  "id": "step_123",
  "tool": "run_command",
  "project_id": "proj_abc",
  "cwd": ".",
  "command": "npm test",
  "timeout_ms": 120000,
  "permission_scope": "project.execute"
}
```

Runner response

```
{
  "id": "step_123",
  "status": "completed",
  "exit_code": 0,
  "stdout": "...",
  "stderr": "",
  "duration_ms": 18422
}
```

Plan object

```
{
  "goal": "Fix login bug",
  "steps": [
    {"id": "1", "description": "Inspect auth code", "tool": "search_files", "risk": "low"},
    {"id": "2", "description": "Apply minimal fix", "tool": "apply_edit", "risk": "project_write"},
    {"id": "3", "description": "Run tests", "tool": "run_command", "risk": "project_execute"}
  ]
}
```

Suggested repository layout

```
curve/<br/> apps/web/<br/> apps/api/<br/> apps/runner/<br/> packages/schemas/<br/>
packages/agent-runtime/<br/> packages/tools/<br/> packages/permissions/<br/>
packages/workflows/<br/> storage/migrations/<br/> tests/runner/<br/>
tests/orchestration/<br/> tests/workflows/
```

28. Final product definition

Curve starts as a local-first AI execution workspace. The MVP is a web application connected to a local Curve Runner. The Runner supplies bounded filesystem, terminal and Git capabilities. Curve supplies planning, orchestration, permissions, approvals, verification, memory, run history and reusable workflows.

The coding agent is the first flagship demonstration, but it is deliberately an agent inside a larger system. Once the core loop is reliable, Curve expands into browser execution, multi-agent workflows, mobile execution and optional cloud execution.

Goal → Project → Agents → Workflow → Permission → Execution Environment → Verification → Artifact → Memory → Reuse

The strategic principle is simple: **Curve should own the orchestration layer, not necessarily every execution machine.** Local compute powers the beginning. Cloud compute can be added when customers justify it. Different models can power intelligence. Different runners can provide execution. Curve remains the system coordinating the pieces.

MVP success condition

A stranger can install/connect Curve, give it a real project task, understand what it intends to do, approve the right actions, watch it execute, inspect what changed, and trust the result enough to use Curve again.

Technical references used for architecture context: OpenAI Agents sandbox documentation and self-hosted environment documentation; Cloudflare coding-agent sandbox documentation. [\[cite\]](#)[\[turn0search0\]](#)[\[turn0search1\]](#)[\[turn0search3\]](#)